Moving a quantum compiler from Python to Rust

Jake Lishman
jake.lishman@ibm.com

IBM

16th of October, 2026

About me

- Work for IBM Research
- Background in physics of quantum computing with trapped ions
- Technical lead of Qiskit, a quantum compiler



https://github.com/Qiskit/qiskit

What is a quantum compiler anyway?

What is a quantum compiler anyway?

Depends on who you ask!

What is a quantum compiler anyway?

Depends on who you ask!

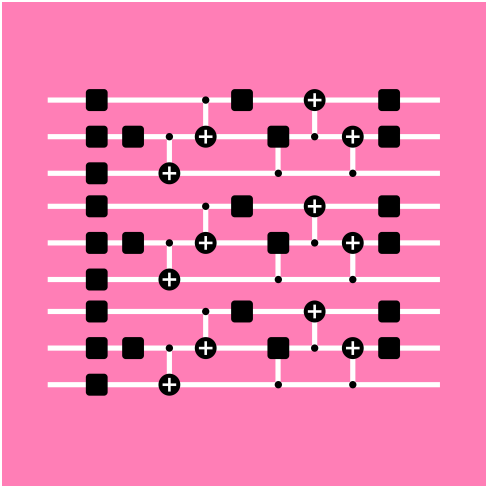
- We'll talk about an early 'gate-model' quantum virtual machine.
- No 'regular' machine code for control hardware to generate.
- No control flow.
- Input programs are already rather low level.
- Rewrite instructions into the supported set.
- Resolve limited connections between qubits.

Qubits, gates and measures

Qubit Both the ‘bit’ and the ‘word’ of quantum data. Also the hardware.

Gate An operation on one or more qubits.

Measure Consume one qubit into a classical bit.



Qubits, gates and measures

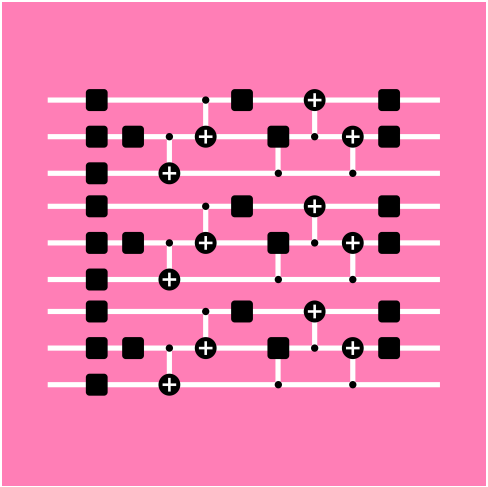
Qubit Both the ‘bit’ and the ‘word’ of quantum data. Also the hardware.

Gate An operation on one or more qubits.

Measure Consume one qubit into a classical bit.

Classical ‘Regular’ computing.

Circuit A program for this virtual machine.



What makes it hard?

- There's no such thing as a "copy".
- There's no such thing as a "store".
- Each qubit can only interact with its immediate neighbours.



Why Python?

```
from qiskit import ...
import some_qpu_provider

backend = some_qpu_provider.backend("ibm_boston")
pm = generate_preset_pass_manager(2, backend)

class MyResearchPass(TransformationPass):
    def run(self, dag): ...

pm.optimization += MyResearchPass()

qc = QuantumCircuit.from_qasm_file("my_prog.qasm")
compiled = pm.run(qc)
```

What compilation looks like

```
OPENQASM 3.0;
include
  "stdgates.inc";

qubit q[5];
bit c[5];

h q[0];
h q[2];
h q[4];
cx q[0], q[1];
cx q[0], q[2];
cx q[0], q[3];
cx q[0], q[4];
c = measure q;
```

What compilation looks like

```
OPENQASM 3.0;
include
  "stdgates.inc";

qubit q[5];
bit c[5];

h q[0];
h q[2];
h q[4];
cx q[0], q[1];
cx q[0], q[2];
cx q[0], q[3];
cx q[0], q[4];
c = measure q;
```

```
OPENQASM 3.0;
include "stdgates.inc";
bit[5] c;
rz(pi/2) $117;
sx $117;
rz(pi) $117;
rz(-pi) $123;
sx $123;
rz(pi/2) $123;
rz(pi/2) $124;
sx $124;
rz(-pi/2) $124;
rz(pi/2) $125;
sx $125;
rz(-3*pi/2) $125;
cz $125, $124;
sx $124;
rz(pi/2) $124;
cz $124, $123;
sx $123;
sx $124;
cz $124, $123;
sx $123;
sx $124;
cz $124, $123;
rz(pi/2) $124;
sx $124;
rz(pi) $124;
cz $125, $126;
cz $125, $117;
sx $117;
cz $125, $124;
sx $124;
rz(pi/2) $126;
sx $126;
c[0] = measure $125;
c[1] = measure $123;
c[2] = measure $126;
c[3] = measure $117;
c[4] = measure $124;
```

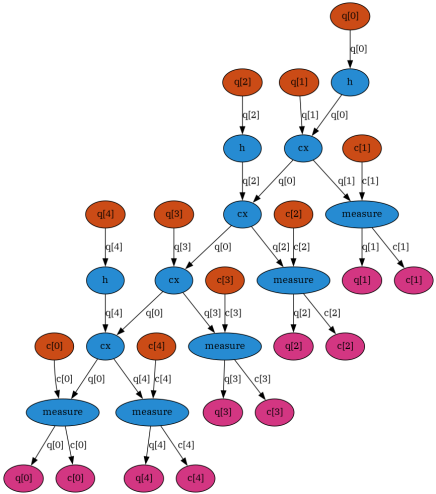
First Rust use: a graph object

```
OPENQASM 3.0;
include "stdgates.inc";

qubit q[5];
bit c[5];

h q[0];
h q[2];
h q[4];
cx q[0], q[1];
cx q[0], q[2];
cx q[0], q[3];
cx q[0], q[4];

c = measure q;
```



Rustworkx

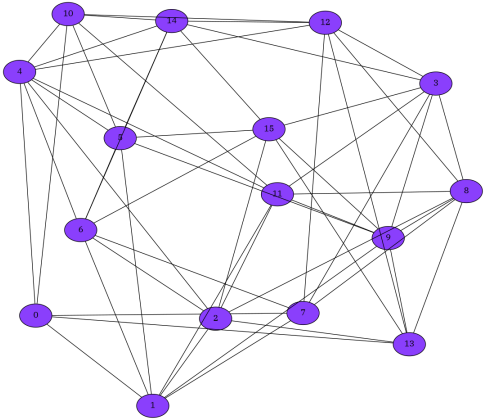
IR manipulations sped up $2\times$

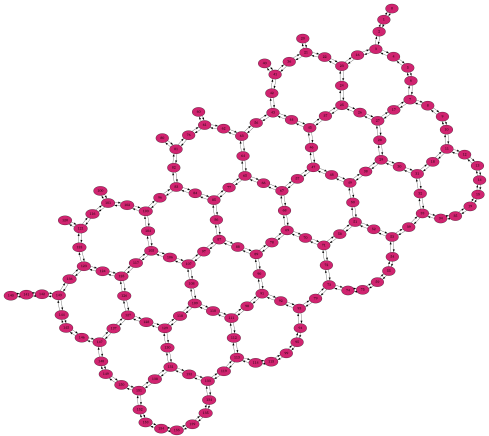
- Wrapper around `petgraph`, exposed to Python with Py03
- General collection of graph algorithms and generators
- Rust bindings in `rustworkx_core`

https://rustworkx.org

https://github.com/Qiskit/rustworkx

The qubit “routing” problem





What makes it hard?

- There's no such thing as a "copy".
- There's no such thing as a "store".
- Each qubit can only interact with its immediate neighbours.

What makes it hard?

- There's no such thing as a "copy".
- There's no such thing as a "store".
- Each qubit can only interact with its immediate neighbours.

Can only shuffle "registers" with the xor swap trick.

```
fn swap(a: &mut u64, b: &mut u64) {
    *a = *a ^ *b;
    *b = *a ^ *b;
    *a = *a ^ *b;
}
```

The qubit “routing” problem

- Quantum computers are noisy; we must minimise swaps
- We use stochastic, heuristic algorithms … so we love threads!

A *lot* of our work is easily parallelised with `rayon` iterators.

Micro-optimisation of core data structures

```
enum Op { Gate(Gate), Matrix(Box<Matrix>), Py(Box<PyOp>) }
#[repr(u8)] enum Gate { I, X, Y, Z, H, CX, RZ, /* ... */ }
```

Micro-optimisation of core data structures

```
enum Op { Gate(Gate), Matrix(Box<Matrix>), Py(Box<PyOp>) }
#[repr(u8)] enum Gate { I, X, Y, Z, H, CX, RZ, /* ... */ }
```

... but `mem::size_of::<Op>() == 16`!

Micro-optimisation of core data structures

```
enum Op { Gate(Gate), Matrix(Box<Matrix>), Py(Box<PyOp>) }
#[repr(u8)] enum Gate { I, X, Y, Z, H, CX, RZ, /* ... */ }
```

... but `mem::size_of::<Op>() == 16`!

```
struct PackedOp(*mut ());

impl PackedOp {
   const MASK: u64 = !0b111u64;

   fn data(&self) -> *mut () {
       self.0.map_addr(|x| x & Self::MASK)
   }
   fn kind(&self) -> OpKind {
       (self.0 as u64 & !Self::MASK).into()
   }
   fn as_ref(&self) -> OpRef<'_> {
       /* ... unsafe shenaningans  ... */
   }
}
```

Our journey

2017 First release of pure-Python Qiskit

2020 Qiskit swaps `NetworkX` for `rustworkx`

2022 Bottleneck routines ported to Rust and `rayon`

2024 Core IR data structures ported to Rust with Py03

2025 Exposed a C API, usable standalone or in Python extensions

2026 Next-gen IRs and compiler are Rust native from day one

What didn't we cover?

- All the complex-number linear algebra (!)
- How we use Py03 to expose a C API to other Python extensions
- The whole field of quantum compilation is changing
- Next-generation compilers mix classical and quantum



https://github.com/Qiskit/qiskit